

DSA: Hashing & Intervals - తెలుగు గైడ్ (LeetCode 150, SSE)

ఈ document చదివిన తర్వాత **Hash Map / Set** మరియు **Intervals** – ఈ రెండు అత్యంత ముఖ్యమైన interview patterns నీ బుర్రలో శాశ్వతంగా నాటుకుపోతాయి. ప్రతి problem కి: ఎలా ఆలోచించాలి (naive నుండి optimal దాకా thought process), brute force ఎందుకు fail అవుతుంది, పైన కనిపించే insight, clean runnable JavaScript solution, step-by-step dry run, complexity reasoning, గుర్తుంచుకోవాల్సిన takeaway, మరియు సాధారణ తప్పులు – అన్నీ ఉంటాయి.

లక్ష్యం: DSA అస్సలు తెలియని వ్యక్తిని కూడా SSE (Senior Software Engineer) interview level కి తీసుకెళ్లడం. "ఒకసారి చదివితే మర్చిపోకూడదు." మనం solutions కంటే ఎలా *think* చేయాలో నేర్చుకుంటాం – ఎందుకంటే interview లో exact problem రాదు, కానీ *pattern* మళ్ళీ మళ్ళీ వస్తుంది.

గమనిక: Time/Space complexity, Big-O notation, arrays, hashing అంటే ఏమిటి, ఎందుకు $O(1)$ – ఇలాంటి fundamentals [DSA_00_Foundations_Telugu.md](#) లో ఉన్నాయి. ఇక్కడ మనం నేరుగా patterns + problems లోకి దూకుతాం. అనుమానం వస్తే foundations doc చూడు.

విషయ సూచిక (Table of Contents)

Patterns (మొదట ఇవి చదవాలి – ఇవే అసలు ఆయుధాలు)

- **Pattern A – Hash Map / Set** ($O(1)$ lookup, frequency counting, seen-set, complement trick, grouping by key)
- **Pattern B – Intervals** (sort by start/end, merge, overlap detection, sweep line, greedy)

Part 1 – Hash Map / Set Problems

1. Ransom Note (LeetCode #383) – Easy
2. Isomorphic Strings (LeetCode #205) – Easy
3. Word Pattern (LeetCode #290) – Easy
4. Valid Anagram (LeetCode #242) – Easy
5. Group Anagrams (LeetCode #49) – Medium
6. Two Sum (LeetCode #1) – Easy
7. Happy Number (LeetCode #202) – Easy
8. Contains Duplicate II (LeetCode #219) – Easy
9. Longest Consecutive Sequence (LeetCode #128) – Medium

Part 2 – Intervals Problems

10. Summary Ranges (LeetCode #228) – Easy
11. Merge Intervals (LeetCode #56) – Medium

12. Insert Interval (LeetCode #57) – Medium

13. Minimum Number of Arrows to Burst Balloons (LeetCode #452) – Medium

Pattern: Hash Map / Set

వివరణ

Hash Map (JavaScript లో **Map** లేదా plain `{}` object) అంటే ఒక **key** → **value** జత లను నిల్వ చేసే data structure. దీని అసలు మ్యాజిక్: ఏ key అయినా **average $O(1)$ time** లో insert / lookup / delete చేయవచ్చు. అంటే – array లో ఒక element ఉందా అని వెతకాలంటే మొత్తం తిరగాలి ($O(n)$), కానీ hash map అడిగితే క్షణంలో చెప్తుంది.

Hash Set (**Set**) అంటే values మాత్రమే ఉన్న map – duplicates ఉండవు, "ఈ item ఇంతకుముందు చూశానా?" అనే ఒక్క ప్రశ్నకి perfect.

ఈ ఒక్క $O(1)$ lookup superpower తో మనం చాలా problems లో **$O(n^2)$ ని $O(n)$ కి** దింపేస్తాం. Hash map అనేది interview లో అత్యధికంగా వాడే pattern – ఐదు రకాల ఉపయోగాలు గుర్తుంచుకో:

ఉపయోగం	ఏం చేస్తుంది	ఉదాహరణ problems
Frequency counting	ప్రతి element ఎన్నిసార్లు వచ్చిందో లెక్కించడం	Anagram, Ransom Note
Seen-set	"ఇది ఇంతకుముందు చూశానా?" $O(1)$ లో	Contains Duplicate, Happy Number
Complement trick	target – current ఉందా అని వెతకడం	Two Sum
Mapping / bijection	ఒక దానికి మరొకటి match అవుతోందా	Isomorphic, Word Pattern
Grouping by key	ఒకే లక్షణం ఉన్నవాటిని కలపడం	Group Anagrams

Real-life Scenario

Array = ఒక పొడవైన train. "5వ bogie లో ఎవరున్నారు?" – index తెలిస్తే వెంటనే చెప్పగలము. కానీ "సూర్య అనే వ్యక్తి ఈ train లో ఉన్నాడా?" అంటే ప్రతి bogie కి వెళ్లి చూడాలి ($O(n)$).

Hash Map = ఒక hotel receptionist దగ్గరున్న register. "సూర్య room ఏది?" అని అడిగితే, receptionist పేజీలు తిప్పుడు – నేరుగా 'S' అక్షరం దగ్గరకి వెళ్లి క్షణంలో చెప్తాడు ($O(1)$). పేరు (key) → room number (value). ఈ receptionist ఎంత పెద్ద hotel అయినా సరే, దాదాపు అదే వేగంతో సమాధానం చెప్తాడు – ఇదే hashing బలం.

Hash function పేరుని (key) ఒక array index గా మారుస్తుంది, అందుకే నేరుగా ఆ చోటుకి jump చేయగలము. అదే "constant time."

ఎలా గుర్తించాలి (Recognition Signals)

ఈ కింది మాటలు / పరిస్థితులు కనిపిస్తే **వెంటనే hash map/set ఆలోచించు**:

- "**duplicate** ఉందా / మళ్ళీ వచ్చిందా / seen before?" → **Set**

- "ఎన్నిసార్లు వచ్చింది / **count / frequency**" → **Map** (frequency counter)
- "ఈ రెండు strings **anagram / permutation** ఆ?" → frequency Map
- "రెండు numbers కలిపితే **target** వస్తుందా?" → **complement trick** (Map)
- "ఒక దానికి మరొకటి **one-to-one map** అవుతోందా?" → రెండు Maps (bijection)
- "**ఒకే రకం** items ని groups గా కలపు" → key → list Map
- నీ brute force **nested loop ($O(n^2)$)** గా ఉంది, లోపలి loop కేవలం "ఇది ఉందా?" అని వెతుకుతోంది → ఆ loop ని Map lookup తో మార్పు → **$O(n)$**

Template Code (JavaScript)

```
// ---- 1. Frequency counter (ప్రతి element ఎన్నిసార్లు) ----
function buildFrequency(arr) {
  const freq = new Map();
  for (const x of arr) {
    freq.set(x, (freq.get(x) || 0) + 1); // లేకపోతే 0, ఉంటే +1
  }
  return freq;
}

// ---- 2. Seen-set (duplicate detection) ----
function hasDuplicate(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true; //  $O(1)$  lookup
    seen.add(x);
  }
  return false;
}

// ---- 3. Complement trick (rough shape) ----
function findPair(arr, target) {
  const seen = new Map(); // value -> index
  for (let i = 0; i < arr.length; i++) {
    const need = target - arr[i]; // ఎంత కావాలో లెక్కించు
    if (seen.has(need)) return [seen.get(need), i];
    seen.set(arr[i], i);
  }
  return null;
}
```

గమనిక — Map vs plain object {} : interview లో **Map** వాడటం safer. కారణం: object keys ఎప్పుడూ strings గా మారతాయి (`obj[1]` మరియు `obj["1"]` ఒకటే), **Map** ఏ type key అయినా (number, string, object) సరిగ్గా ఉంచుతుంది. అదనంగా **Map** లో `.size`, సులభమైన iteration, మరియు `hasOwnProperty` గొడవ ఉండదు.

Complexity

- **Lookup / insert / delete:** average **$O(1)$** , worst case $O(n)$ (అన్ని keys ఒకే bucket లో collide అయితే – practical గా అరుదు).
- **మొత్తం array ని ఒకసారి process చేస్తే: $O(n)$ time.**
- **Space:** worst case **$O(n)$** – n distinct keys map లో పెట్టాల్సి రావచ్చు.
- **Trade-off:** extra memory (space) ఖర్చు పెట్టి, time ని $O(n^2)$ నుండి $O(n)$ కి కొంటున్నాం. ఇదే classic **"space-time trade-off."**

1. Ransom Note (LeetCode #383) – Easy

సమస్య

రెండు strings ఇస్తారు: ransomNote మరియు magazine . magazine లోని అక్షరాలను (letters) కట్టిరించి ransomNote ని తయారు చేయగలమా అని చెప్పాలి. **షరతు:** magazine లోని ప్రతి అక్షరాన్ని **ఒక్కసారి మాత్రమే** వాడగలం.

Constraints: $1 \leq \text{ransomNote.length}, \text{magazine.length} \leq 10^5$; రెండూ చిన్న English letters (a - z) మాత్రమే.

ఉదాహరణ

```
Input:  ransomNote = "aa", magazine = "aab"
Output: true           // 'a' రెండుసార్లు magazine లో ఉంది, సరిపోతుంది

Input:  ransomNote = "aa", magazine = "ab"
Output: false          // 'a' ఒక్కటే ఉంది, రెండు కావాలి → false
```

ఎలా ఆలోచించాలి

అసలు ప్రశ్న ఏమిటి? – "ransomNote లో కావాల్సిన ప్రతి అక్షరం, magazine లో **సరిపడా సంఖ్యలో** ఉందా?" ఇది ఒక **counting / inventory** సమస్య.

మొదటి ఆలోచన (naive): ransomNote లోని ప్రతి అక్షరానికీ, magazine లో వెతికి, దొరికితే ఆ అక్షరాన్ని magazine నుండి "తీసేయడం" (మళ్ళీ వాడకుండా). కానీ ప్రతిసారి magazine మొత్తం వెతకడం ఖరీదు ($O(n \times m)$).

Insight: magazine లో ఏ అక్షరం ఎన్నిసార్లు ఉందో ఒక్కసారి **లెక్కపెట్టేసుకో** (inventory తయారు చేయ్). తర్వాత ransomNote లోని ప్రతి అక్షరానికీ ఆ inventory నుండి ఒకటి "తీసేయ్". ఏ అక్షరం stock అయిపోతే (0 అయితే) → **false** . ఇది **frequency counting** pattern.

Brute Force

ransomNote లోని ప్రతి character కి magazine లో linear search చేసి, దొరికిన character ని ఒక marker తో "వాడేశాం" అని గుర్తు పెట్టడం.

```
function canConstructBrute(ransomNote, magazine) {
  const mag = magazine.split(''); // mutate చేయగలిగేలా array
  for (const ch of ransomNote) {
    const idx = mag.indexOf(ch); // O(m) search
    if (idx === -1) return false; // ఎక్కడా దొరకలేదు
    mag[idx] = null; // ఈ copy ని వాడేశాం
  }
  return true;
}
```

- **Time:** $O(n \times m)$ — ransomNote లోని n అక్షరాలకి, ప్రతిదానికి magazine లో $O(m)$ search.
- **సమస్య:** n, m రెండూ 10^5 దాకా వెళ్తే $\sim 10^{10}$ operations — చాలా నెమ్మది, TLE (Time Limit Exceeded).

Optimal Approach

Insight: ప్రతిసారి వెతకడం బదులు, magazine ని **ఒక్కసారి** scan చేసి frequency map తయారు చేయ్: ఏ అక్షరం ఎన్నిసార్లు ఉంది. తర్వాత ransomNote లోని ప్రతి అక్షరానికి map నుండి ఒకటి తీసేయ్. count 0 కి పడిపోతే → తయారు చేయలేం.

Plan:

1. magazine scan చేసి **count** map build చేయ్ (letter → ఎన్నిసార్లు).
2. ransomNote లోని ప్రతి ch కి: **count[ch]** 0 లేదా లేకపోతే → **false**. లేదంటే **count[ch]--**.
3. అన్నీ దాటితే → **true**.

Pattern: **Hash Map — frequency counting.**

Solution (JavaScript)

```
/**
 * @param {string} ransomNote
 * @param {string} magazine
 * @return {boolean}
 */
function canConstruct(ransomNote, magazine) {
  // Chinna optimization: note magazine kanna peddadaithe eppatiki false
  if (ransomNote.length > magazine.length) return false;

  // Step 1: magazine lo prati letter frequency lekkinchu
  const count = new Map();
  for (const ch of magazine) {
    count.set(ch, (count.get(ch) || 0) + 1);
  }

  // Step 2: ransomNote lo prati letter ki stock nunchi okati teeseey
  for (const ch of ransomNote) {
    const available = count.get(ch) || 0;
    if (available === 0) return false; // stock aipoyindi
    count.set(ch, available - 1); // okati vaadesam
  }
}
```

```
    return true; // anni letters saripoyayi
}
```

Dry Run

ransomNote = "aa" , magazine = "aab" :

1. **Build count** magazine "aab" నుండి $\rightarrow \{a: 2, b: 1\}$.
2. ransomNote[0] = 'a' : available = 2 $\rightarrow$ 0 కాదు $\rightarrow$ count ఇప్పుడు $\{a: 1, b: 1\}$.
3. ransomNote[1] = 'a' : available = 1 $\rightarrow$ 0 కాదు $\rightarrow$ count ఇప్పుడు $\{a: 0, b: 1\}$.
4. Loop అయిపోయింది $\rightarrow$ return true. ✓

ransomNote = "aa" , magazine = "ab" :

1. **Build count** $\rightarrow \{a: 1, b: 1\}$.
2. ransomNote[0] = 'a' : available = 1 $\rightarrow \{a: 0, b: 1\}$.
3. ransomNote[1] = 'a' : available = 0 $\rightarrow$ return false. ✓

Complexity

- **Time: $O(n + m)$.** magazine ని ఒక్కసారి scan (m), ransomNote ని ఒక్కసారి scan (n). Map operations $O(1)$.
- **Space: $O(k)$** – k = distinct characters. ఇక్కడ letters a – z కాబట్టి $k \leq 26 \rightarrow$ practically **$O(1)$** .

గుర్తుంచుకోవాల్సినది

- "ఒక collection లోని items మరో collection తయారుచేయడానికి సరిపోతాయా / cover చేస్తాయా?" $\rightarrow$ frequency map వెయ్, తర్వాత తీసేయ్.
- ఇదే idea Valid Anagram (#242) లో కూడా – తేడా ఏమిటంటే anagram లో count సరిగ్గా సమానం కావాలి, ఇక్కడ magazine లో ఎక్కువ ఉన్నా పర్వాలేదు.
- `count.get(ch) || 0` అనే idiom – "లేకపోతే 0" – frequency maps లో ప్రతిచోటా వాడతావు.

సాధారణ తప్పులు

- **Early length check మర్చిపోవడం:** అవసరం లేదు కానీ `ransomNote.length > magazine.length` అయితే వెంటనే false – చిన్న speedup.
- `count.get(ch)` undefined ని handle చేయకపోవడం: map లో లేని key కి `.get` $\rightarrow$ undefined . `|| 0` పెట్టకపోతే `undefined - 1 = NaN` వచ్చి logic పాడవుతుంది.
- magazine ని కాకుండా **ransomNote ని count చేయడం** పొరపాటు – direction ముఖ్యం: మనం magazine stock నుండి తీసేస్తాం.
- Uppercase/lowercase కలిస్తే (ఈ problem లో అన్నీ lowercase, కానీ variants లో) 'A' $\neq$ 'a' గుర్తుంచుకో.

2. Isomorphic Strings (LeetCode #205) — Easy

సమస్య

రెండు strings s మరియు t ఇస్తారు. అవి **isomorphic** ఆ అని చెప్పాలి. Isomorphic అంటే — s లోని అక్షరాలను ఒక **స్థిరమైన నియమం** ప్రకారం మార్చి (replace) t ని పొందగలగాలి. షరతులు రెండు: (1) ఒక అక్షరం క్రమం (order) చెడకూడదు, (2) s లోని రెండు వేర్వేరు అక్షరాలు t లో **ఒకే** అక్షరానికి map అవుతూ ఉండదు, కానీ ఒక అక్షరం తనను తానే map చేసుకోవచ్చు.

Constraints: $1 \leq s.length \leq 5 * 10^4$; $t.length == s.length$; అన్నీ printable ASCII characters.

ఉదాహరణ

```
Input: s = "egg", t = "add" → Output: true // e→a, g→d (స్థిరం)
Input: s = "foo", t = "bar" → Output: false // o→a, అప్పుడు o→r? ఒకే o రెండు map → false
Input: s = "paper", t = "title" → Output: true // p→t, a→i, e→l, r→e
Input: s = "badc", t = "baba" → Output: false // d,c రెండూ 'a' కి map → clash
```

ఎలా ఆలోచించాలి

"Replace to get t" అంటే s లోని ప్రతి అక్షరానికి t లో ఒక **జోడీ** (partner) ఉండాలి — అదే జోడీ ప్రతినారి. అంటే ఒక **mapping** (s -అక్షరం $\rightarrow$ t -అక్షరం) ఉండాలి, అది consistent గా ఉండాలి.

కానీ ఒక్క direction సరిపోదు. ఉదా. $s="badc", t="baba" : b \rightarrow b, a \rightarrow a$ వరకు బాగుంది, కానీ $d \rightarrow a$ మరియు $c \rightarrow a$ — d, c రెండూ 'a' కి map అవుతున్నాయి. ఇది isomorphic కాదు (రెండు source అక్షరాలు ఒకే target కి). కాబట్టి mapping **one-to-one (bijection)** అవ్వాలి — అందుకే **రెండు వైపులా** check చేయాలి: $s \rightarrow t$ మరియు $t \rightarrow s$.

Insight: రెండు hash maps పెట్టు. ప్రతి position i వద్ద, $s[i] \leftrightarrow t[i]$ జోడీ ఇప్పటికే ఏర్పడిన నియమానికి విరుద్ధంగా లేదని రెండు వైపులా నిర్ధారించు.

Brute Force

ప్రతి జత positions (i, j) చూసి consistency check: $s[i] == s[j]$ అయితే $t[i] == t[j]$ అవ్వాలి, మరియు t Reverse కూడా.

```
function isIsomorphicBrute(s, t) {
  if (s.length !== t.length) return false;
  for (let i = 0; i < s.length; i++) {
    for (let j = i + 1; j < s.length; j++) {
      // s vaipu same aithe t vaipu kooda same avvali (mariyu reverse)
      if ((s[i] === s[j]) !== (t[i] === t[j])) return false;
    }
  }
  return true;
}
```

- **Time:** $O(n^2)$ — ప్రతి జత positions.

- **సమస్య:** $n = 5 \times 10^4$ అయితే $\sim 10^9$ operations $\rightarrow$ TLE. అవసరం లేని పని.

Optimal Approach

Insight: ప్రతి జతని పోల్చడం బదులు, "ఇప్పటివరకూ ఏర్పడిన mapping" ని hash map లో గుర్తుంచుకో. కొత్త position వచ్చినప్పుడు ఆ mapping కి విరుద్ధంగా ఉందా అని $O(1)$ లో check చెయ్.

Plan:

1. mapST ($s \rightarrow t$), mapTS ($t \rightarrow s$) రెండు maps.
2. ప్రతి i కి $a = s[i]$, $b = t[i]$:
 - mapST లో **a** ఉంది కానీ $\text{mapST}[a] \neq b \rightarrow \text{clash} \rightarrow \text{false}$.
 - mapTS లో **b** ఉంది కానీ $\text{mapTS}[b] \neq a \rightarrow \text{clash} \rightarrow \text{false}$.
 - లేదంటే రెండు maps లో set చెయ్.
3. అన్నీ దాటితే $\rightarrow \text{true}$.

Pattern: **Hash Map — bijection (two-way mapping).**

Solution (JavaScript)

```
/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
function isIsomorphic(s, t) {
    if (s.length !== t.length) return false;

    const mapST = new Map(); // s-akshram -> t-akshram
    const mapTS = new Map(); // t-akshram -> s-akshram (reverse, bijection kosam)

    for (let i = 0; i < s.length; i++) {
        const a = s[i];
        const b = t[i];

        // s vaipu: 'a' ki idivaraku vere partner unte clash
        if (mapST.has(a) && mapST.get(a) !== b) return false;
        // t vaipu: 'b' ki idivaraku vere source unte clash
        if (mapTS.has(b) && mapTS.get(b) !== a) return false;

        mapST.set(a, b);
        mapTS.set(b, a);
    }

    return true;
}
```

Dry Run

$s = \text{"badc"} , t = \text{"baba"} :$

i	a=s[i]	b=t[i]	mapST check	mapTS check	action
0	b	b	కొత్తది	కొత్తది	set b→b, b→b
1	a	a	కొత్తది	కొత్తది	set a→a, a→a
2	d	b	కొత్తది	mapTS[b] = b ≠ d → clash!	return false ✓

s = "paper" , t = "title" : p→t, a→i, p→t (మళ్ళీ, consistent), e→l, r→e — ఏ clash లేదు → **true** . ✓

Complexity

- **Time: O(n)**. ఒక్క pass, ప్రతి step O(1) map operations.
- **Space: O(k)**. k = distinct characters (ASCII అయితే ≤ 128 → O(1)). రెండు maps.

గుర్తుంచుకోవాల్సినది

- "ఒక దానికి మరొకటి **one-to-one match అవుతోందా?**" → **రెండు maps** (bijection). ఒక్క map అయితే "many-to-one" clash పట్టుకోలేవు.
- Word Pattern (#290) ఇదే problem — తేడా: characters బదులు **words**. Same రెండు-maps trick.
- ఒక చిన్న ట్రిక్: string ని "first-occurrence index pattern" గా encode చేసి పోల్చడం కూడా పని చేస్తుంది (egg → **011** , add → **011**), కానీ two-map approach చదవడానికి, చెప్పడానికి clear.

సాధారణ తప్పులు

- **ఒక్క map మాత్రమే** వాడటం — అప్పుడు "badc" → "baba" కి తప్పుగా **true** వస్తుంది (d→a, c→a clash పట్టుకోలేదు). రెండు maps తప్పనిసరి.
- Length check మర్చిపోవడం (ఇక్కడ constraint t.length == s.length అయినా habit గా పెట్టు).
- mapST.get(a) !== b లో has check లేకుండా నేరుగా compare చేస్తే — key లేనప్పుడు **undefined** !== b → true → తప్పుగా false వస్తుంది. has ముందు check చెయ్యి.

3. Word Pattern (LeetCode #290) — Easy

సమస్య

ఒక **pattern** (letters string, ఉదా. "abba") మరియు ఒక s (space తో విడదీసిన words, ఉదా. "dog cat cat dog") ఇస్తారు. s అనేది **pattern** ని **follow అవుతోందా** అని చెప్పాలి. Follow అంటే — pattern లోని ప్రతి letter కి, s లోని ఒక word తో **bijection** (one-to-one, రెండు వైపులా) ఉండాలి.

Constraints: 1 ≤ pattern.length ≤ 300 ; pattern లో lowercase letters; s లో lowercase words + single spaces; leading/trailing space లేదు.

ఉదాహరణ

```
pattern = "abba", s = "dog cat cat dog" → true // a->dog, b->cat
pattern = "abba", s = "dog cat cat fish" → false // a->dog కానీ చివర a->fish clash
pattern = "aaaa", s = "dog cat cat dog" → false // a అన్నిటికీ కానీ words వేరు
pattern = "abba", s = "dog dog dog dog" → false // a,b రెండూ dog కి map → clash
```

ఎలా ఆలోచించాలి

ఇది **Isomorphic Strings** కి కవల సోదరుడు (twin). తేడా ఒక్కటే: అక్కడ character ↔ character, ఇక్కడ character ↔ word. మిగతా logic అదే – ఒక **consistent one-to-one mapping** ఉందా అని రెండు వైపులా చూడాలి.

ముందు s ని words గా విడదీయాలి (`split(' ')`). తర్వాత `pattern.length` మరియు `words.length` సమానం కాకపోతే వెంటనే **false** (ప్రతి letter కి ఒక word కావాలి కదా).

Insight: `char` → `word` (map1) మరియు `word` → `char` (map2). ప్రతి జతని రెండు maps తో verify చెయ్ – Isomorphic లాగే.

Brute Force

`pattern` లోని ప్రతి జత positions (i, j) కి: `pattern[i]==pattern[j]` అయితే `words[i]==words[j]` అవ్వాలి (మరియు reverse).

```
function wordPatternBrute(pattern, s) {
  const words = s.split(' ');
  if (pattern.length !== words.length) return false;
  for (let i = 0; i < pattern.length; i++) {
    for (let j = i + 1; j < pattern.length; j++) {
      if ((pattern[i] === pattern[j]) !== (words[i] === words[j])) return false;
    }
  }
  return true;
}
```

- **Time:** $O(n^2 \times L)$ – n = words, L = word compare cost.
- **సమస్య:** అవసరం లేని $O(n^2)$. Hash map తో $O(n)$ కి తగ్గించవచ్చు.

Optimal Approach

Insight: "ఇప్పటివరకూ ఏర్పడిన mapping" ని రెండు maps లో ఉంచి, కొత్త జత వచ్చినప్పుడు $O(1)$ లో verify చెయ్.

Plan:

1. `words = s.split(' ')` . `pattern.length !== words.length` → **false** .
2. `charToWord` , `wordToChar` maps.
3. ప్రతి i కి `c = pattern[i]` , `w = words[i]` :
 - `charToWord[c]` ఉంది కానీ `!== w` → **false** .
 - `wordToChar[w]` ఉంది కానీ `!== c` → **false** .

- రెండూ set చెయ్యి.

4. `true` .

Pattern: **Hash Map — bijection.**

Solution (JavaScript)

```
/**
 * @param {string} pattern
 * @param {string} s
 * @return {boolean}
 */
function wordPattern(pattern, s) {
    const words = s.split(' ');
    // Prati letter ki oka word - counts saripovali
    if (pattern.length !== words.length) return false;

    const charToWord = new Map();
    const wordToChar = new Map();

    for (let i = 0; i < pattern.length; i++) {
        const c = pattern[i];
        const w = words[i];

        if (charToWord.has(c) && charToWord.get(c) !== w) return false;
        if (wordToChar.has(w) && wordToChar.get(w) !== c) return false;

        charToWord.set(c, w);
        wordToChar.set(w, c);
    }

    return true;
}
```

Dry Run

pattern = "abba" , s = "dog cat cat fish" → words = ["dog","cat","cat","fish"] :

i	c	w	charToWord	wordToChar	action
0	a	dog	కొత్తది	కొత్తది	a→dog, dog→a
1	b	cat	కొత్తది	కొత్తది	b→cat, cat→b
2	b	cat	b→cat ✓	cat→b ✓	ok
3	a	fish	charToWord[a] =dog ≠ fish → clash	—	return false ✓

Complexity

- **Time: $O(n + m)$.** n = pattern length, m = s length (split కోసం). ప్రతి step $O(1)$ (word comparison average $O(L)$ కానీ practically small).

- **Space: $O(n)$.** words array + రెండు maps.

గుర్తుంచుకోవాల్సినది

- Isomorphic Strings ↔ Word Pattern — **ఒకే pattern, రెండు రూపాలు.** character↔character vs character↔word. bijection = రెండు maps.
- **Length mismatch వెంటనే false** — ఇది ఇక్కడ కీలకమైన edge case (interviewers దీన్ని పరీక్షిస్తారు).
- ఏదైనా "X ↔ Y consistent one-to-one match" అడిగితే → ఈ two-map skeleton వాడు.

సాధారణ తప్పులు

- **Length check మర్చిపోవడం:** pattern="aaa", s="dog dog" — length mismatch. Check లేకపోతే loop pattern length దాకా వెళ్లి `words[i]` undefined అవుతుంది → తప్పు.
- **ఒక్క map:** pattern="abba", s="dog dog dog dog" కి తప్పుగా true వస్తుంది (a→dog, b→dog రెండూ dog కి). రెండు maps కావాలి.
- `split(' ')` బదులు `split(/\s+/)` వాడితే multiple spaces ఉన్న edge cases లో word counts మారిపోవచ్చు — ఈ problem లో single space guarantee ఉంది కాబట్టి `split(' ')` సరి.

4. Valid Anagram (LeetCode #242) — Easy

సమస్య

రెండు strings `s`, `t` ఇస్తారు. `t` అనేది `s` యొక్క **anagram** ఆ అని చెప్పాలి. Anagram అంటే — అవే అక్షరాలు, అవే సంఖ్యలో, కానీ వేరే క్రమంలో (rearrangement). ఉదా. "listen" మరియు "silent".

Constraints: `1 <= s.length, t.length <= 5 * 104` ; lowercase English letters.

ఉదాహరణ

```
s = "anagram", t = "nagaram" → true
s = "rat",      t = "car"      → false // 't' vs 'c'/'r' — akshraḷu verugga
```

ఎలా ఆలోచించాలి

Anagram అంటే "అవే అక్షరాలు, అదే count." కాబట్టి ప్రశ్న: **రెండు strings యొక్క letter frequency ఒకటేనా?**

రెండు సులభమైన దారులు:

1. **Sort చేసి పోల్చడం:** రెండింటినీ sort చేస్తే anagrams identical అవుతాయి (`"rat".sort() = "art"`, `"car".sort()="acr"` → వేరు). Simple కానీ $O(n \log n)$.
2. **Frequency count:** `s` లోని అక్షరాలు లెక్కపెట్టి, `t` లోని వాటిని తీసేయ్ — Ransom Note లాగే. ఏదైనా negative అయితే / మిగిలితే → కాదు. $O(n)$.

Insight: Length వేరైతే వెంటనే false. లేదంటే frequency map — Ransom Note కి దగ్గరి బంధువు, కానీ ఇక్కడ counts సరిగ్గా సమానం కావాలి (ఎక్కువ ఉంటే కూడా false).

Brute Force

s లోని ప్రతి character కి t లో ఒక instance వెతికి తీసేయడం.

```
function isAnagramBrute(s, t) {
  if (s.length !== t.length) return false;
  const tArr = t.split('');
  for (const ch of s) {
    const idx = tArr.indexOf(ch); // O(n) search
    if (idx === -1) return false;
    tArr.splice(idx, 1);          // teesey (mari O(n) shift)
  }
  return tArr.length === 0;
}
```

- **Time:** $O(n^2)$ — ప్రతి character కి search + splice.
- **సమస్య:** పెద్ద inputs కి నెమ్మది. Frequency map తో $O(n)$.

Optimal Approach

Insight: ఒకే frequency map వాడు: s కి +1, t కి -1. చివర్లో అన్ని counts 0 అయితే → anagram.

Plan:

1. Length వేరైతే → false.
2. s కి count[ch]++.
3. t కి count[ch]--; ఏదైనా 0 కంటే తక్కువకి పడిపోతే (t లో ఆ letter ఎక్కువ) → false.
4. అన్నీ దాటితే → true (lengths సమానం కాబట్టి leftover positives కూడా ఉండవు).

Pattern: **Hash Map — frequency counting.**

Solution (JavaScript)

```
/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
function isAnagram(s, t) {
  // Lengths vere aithe eppatiki anagram kaadu
  if (s.length !== t.length) return false;

  const count = new Map();

  // s lo prati letter ki +1
  for (const ch of s) {
    count.set(ch, (count.get(ch) || 0) + 1);
  }
}
```

```

}

// t lo prati letter ki -1
for (const ch of t) {
  const c = count.get(ch) || 0;
  if (c === 0) return false; // t lo ee letter ekkuva undi
  count.set(ch, c - 1);
}

return true; // anni counts 0 (lengths samanam kabatti guarantee)
}

```

26-size array trick (bonus): అన్నీ `a - z` అయినప్పుడు Map బదులు 26-length array వాడొచ్చు – `count[ch.charCodeAt(0) - 97]`. కొంచెం fast మరియు constant space. కానీ Map version generic (Unicode కి కూడా పని చేస్తుంది).

Dry Run

`s = "anagram"` , `t = "nagaram"` (both length 7):

1. Count `s` : `a:3`, `n:1`, `g:1`, `r:1`, `m:1`.
2. Process `t = "nagaram"` : `n`→(1→0), `a`→(3→2), `g`→(1→0), `a`→(2→1), `r`→(1→0), `a`→(1→0), `m`→(1→0). ఏదీ 0 దగ్గర fail కాలేదు.
3. `return true` . ✓

`s = "rat"` , `t = "car"` :

1. Count `s` : `r:1`, `a:1`, `t:1`.
2. Process `t = "car"` : `c`→ `count.get('c') = 0` → `return false` . ✓

Complexity

- **Time: $O(n)$.** రెండు linear passes (`s` , `t`), $O(1)$ map ops.
- **Space: $O(k)$,** k = distinct letters $\leq 26 \rightarrow O(1)$.

గుర్తుంచుకోవాల్సినది

- "అవే elements అదే count లో ఉన్నాయా?" → single frequency map, +1 అప్పుడు -1, చివర్లో అన్నీ 0.
- Length mismatch = instant false. ఈ ఒక్క line చాలా bugs కాపాడుతుంది.
- Group Anagrams (#49) ఇదే idea పైన కట్టబడింది – అక్కడ "anagram signature" ని key గా వాడతాం.
- Sort-based solution ($O(n \log n)$) కూడా చెప్పు – interviewer trade-off అడగవచ్చు (code short కానీ slower).

సాధారణ తప్పులు

- **Length check మర్చిపోవడం** – `"a"` vs `"ab"` లాంటివి తప్పుగా pass అవుతాయి.

- **t లోని decrement లో 0 check మర్చిపోవడం** — `c === 0` చూడకపోతే negative counts వచ్చి logic పాడవుతుంది.
- **Unicode / multi-byte characters:** `charCodeAt` array trick ASCII కి మాత్రమే safe. Emoji, accented letters ఉంటే Map వాడు (లేదా `for...of` వాడు, ఇది code points ని సరిగ్గా iterate చేస్తుంది; `for + index` UTF-16 surrogate pairs ని విడగొడుతుంది).

5. Group Anagrams (LeetCode #49) — Medium

సమస్య

Strings యొక్క array `strs` ఇస్తారు. అందులో ఒకదానికొకటి **anagrams** అయిన వాటిని ఒకే group లో పెట్టి, groups యొక్క list return చేయాలి. Groups క్రమం ఏదైనా సరే.

Constraints: `1 <= strs.length <= 10^4`; `0 <= strs[i].length <= 100`; lowercase English letters.

ఉదాహరణ

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`
Output: `[["eat","tea","ate"], ["tan","nat"], ["bat"]]`

ఎలా ఆలోచించాలి

Anagrams అన్నిటికీ **ఉమ్మడి లక్షణం** ఏమిటి? — sort చేస్తే అవన్నీ **ఒకటే string** అవుతాయి. `"eat", "tea", "ate"` అన్నీ sort చేస్తే `"aet"`. అంటే `"aet"` వీటన్నిటి **signature / fingerprint**.

Insight: ఒకే signature ఉన్నవాటిని కలపాలంటే — signature ని **hash map key** గా వాడి, ఆ key కింద strings ని list గా జోడించు. ఇది **grouping by key** pattern.

రెండు రకాల signatures:

1. **Sorted string:** `str.split('').sort().join('')` — తయారు చేయడానికి $O(k \log k)$.
2. **Count signature:** ప్రతి letter count ని encode చెయ్య (`a1b0c0...` లేదా 26-length count array ని string గా). $O(k)$ — sort అవసరం లేదు, పెద్ద strings కి fast.

Brute Force

ప్రతి string ని ప్రతి existing group యొక్క representative తో "anagram అ?" అని పోల్చడం.

```
function groupAnagramsBrute(strs) {
  const groups = [];
  for (const s of strs) {
    let placed = false;
    for (const g of groups) {
      if (isAnagram(s, g[0])) { g.push(s); placed = true; break; }
    }
    if (!placed) groups.push([s]);
  }
}
```

```

    }
    return groups;
}

```

- **Time:** $O(n^2 \times k)$ – ప్రతి string కి అన్ని groups తో anagram check.
- **సమస్య:** $n = 10^4$ అయితే $10^8 \times k \rightarrow$ నెమ్మది. Grouping ని hash తో $O(1)$ చేయవచ్చు.

Optimal Approach

Insight: ప్రతి string యొక్క signature లెక్కించి, signature $\rightarrow$ list map లో పడేయ్. అదే signature ఉన్నవి automatic గా ఒకే bucket లోకి వస్తాయి.

Plan:

1. groups = new Map() .
2. ప్రతి str కి: key = signature(str) (sorted లేదా count).
3. groups[key] లేకపోతే [] పెట్టు, ఆపై str ని push.
4. చివర్లో [...groups.values()] .

Pattern: **Hash Map – grouping by key.**

Solution (JavaScript)

```

/**
 * @param {string[]} strs
 * @return {string[][]}
 */
function groupAnagrams(strs) {
    const groups = new Map(); // signature -> [words...]

    for (const str of strs) {
        // Signature: akshralanu sort chesi join - anagrams anni okate key
        const key = str.split('').sort().join('');

        if (!groups.has(key)) groups.set(key, []);
        groups.get(key).push(str);
    }

    return [...groups.values()];
}

// ---- Alternative: count-based key (O(n*k), sort avasaram ledu) ----
function groupAnagramsCount(strs) {
    const groups = new Map();
    for (const str of strs) {
        const count = new Array(26).fill(0);
        for (const ch of str) count[ch.charCodeAt(0) - 97]++;
        const key = count.join(','); // "1,0,0,...,2" - unique per multiset
        if (!groups.has(key)) groups.set(key, []);
        groups.get(key).push(str);
    }
}

```

```
return [...groups.values()];
}
```

Dry Run

`strs = ["eat","tea","tan","ate","nat","bat"]` (sorted-key version):

str	key (sorted)	groups state
eat	aet	{aet: [eat]}
tea	aet	{aet: [eat,tea]}
tan	ant	{aet:[...], ant:[tan]}
ate	aet	{aet:[eat,tea,ate], ant:[tan]}
nat	ant	{aet:[...], ant:[tan,nat]}
bat	abt	{aet:[...], ant:[...], abt:[bat]}

`groups.values()` → `[["eat","tea","ate"], ["tan","nat"], ["bat"]]` . ✓

Complexity

Let n = strings, k = max string length.

- **Sorted-key: Time $O(n \cdot k \log k)$** – ప్రతి string ని sort. **Space $O(n \cdot k)$** – అన్ని strings map లో.
- **Count-key: Time $O(n \cdot k)$** – sort లేదు, letter counting $O(k)$. Space same $O(n \cdot k)$.
- k పెద్దగా ఉంటే count-key faster. చిన్న strings కి sorted-key చాలు (code short).

గుర్తుంచుకోవాల్సినది

- "ఒకే లక్షణం ఉన్నవాటిని groups గా కలపు" → ఆ లక్షణాన్ని **canonical key** గా మార్చి `Map<key, list>` వాడు. ఇది universal grouping trick (anagrams, ఒకే sum, ఒకే remainder, etc.).
- Signature డిజైన్ ముఖ్యం: ఏ రెండు items ఒకే group లో ఉండాలంటే ఒకే signature, వేరు అయితే వేరు signature ఇవ్వాలి.
- `[...map.values()]` (లేదా `Array.from(map.values())`) – Map ని array of lists గా మార్చే idiom.

సాధారణ తప్పులు

- **Count-key లో separator మర్చిపోవడం:** `count.join('')` వాడితే `[1,12]` మరియు `[11,2]` రెండూ `"112"` అవుతాయి → wrong collision. `join(',')` (లేదా `#`) తో separate చెయ్యి.
- **Empty string** (`""`): valid input, దాని key `""` – code సహజంగా handle చేస్తుంది, కానీ మర్చిపోకు.
- Sort ఒరిజినల్ string ని mutate చేయదు (`split` కొత్త array ఇస్తుంది) – కానీ నేరుగా array మీద sort చేస్తే original పోతుంది, గుర్తుంచుకో.
- Object `{}` ని map గా వాడితే insertion order integer-like keys కి మారిపోవచ్చు – **Map** వాడటం safe.

6. Two Sum (LeetCode #1) — Easy

సమస్య

Integers array `nums` మరియు ఒక `target` ఇస్తారు. కలిపితే `target` వచ్చే రెండు elements యొక్క `indices` return చేయ్. సరిగ్గా ఒకే ఒక సమాధానం ఉంటుందని guarantee; ఒకే element ని రెండుసార్లు వాడకూడదు.

Constraints: $2 \leq \text{nums.length} \leq 10^4$; $-10^9 \leq \text{nums}[i], \text{target} \leq 10^9$; ఖచ్చితంగా ఒక solution.

ఉదాహరణ

```
nums = [2,7,11,15], target = 9 → [0,1] // 2+7=9
nums = [3,2,4], target = 6 → [1,2] // 2+4=6
nums = [3,3], target = 6 → [0,1]
```

ఎలా ఆలోచించాలి

ప్రతి element `x` కి, దాని జేడ్ ఎంత ఉండాలి? — `target - x`. దీన్ని **complement** అంటారు. కాబట్టి ప్రశ్న: "ప్రతి `x` కి, `target - x` అనే number array లో ఉందా?"

Naive గా ప్రతి `x` కి array మొత్తం వెతికితే $O(n^2)$. కానీ "ఈ number ఉందా, ఉంటే ఏ index లో?" — ఇది hash map కి perfect ప్రశ్న ($O(1)$).

Insight: array ని ఒక్కసారి తిరుగుతూ, ఇప్పటివరకూ చూసిన numbers ని `value → index` map లో ఉంచు. ప్రస్తుత `x` వచ్చినప్పుడు, `target - x` ఇప్పటికే map లో ఉందా అని చూడు — ఉంటే జేడ్ దొరికింది! ఇది **complement trick** — one-pass.

Brute Force

అన్ని జతలు (`i, j`) చూడటం.

```
function twoSumBrute(nums, target) {
  for (let i = 0; i < nums.length; i++) {
    for (let j = i + 1; j < nums.length; j++) {
      if (nums[i] + nums[j] === target) return [i, j];
    }
  }
  return [];
}
```

- **Time:** $O(n^2)$ — అన్ని జతలు.
- **Space:** $O(1)$.
- **సమస్య:** $n = 10^4 \rightarrow 10^8$ operations. పని చేస్తుంది కానీ నెమ్మది; hash map తో $O(n)$.

Optimal Approach

Insight: వెతకడాన్ని $O(1)$ map lookup గా మార్చు. Sorting అవసరం లేదు (అది indices ని పాడు చేస్తుంది). One-pass సరిపోతుంది – element ని map లో పెట్టేముందు దాని complement ని check చేస్తాం, కాబట్టి same element రెండుసార్లు వాడబడదు.

Plan:

1. `seen = new Map()` (value $\rightarrow$ index).
2. ప్రతి i కి: `need = target - nums[i]` .
 - `seen.has(need)  $\rightarrow$  return [seen.get(need), i]` .
 - లేదంటే `seen.set(nums[i], i)` .

Pattern: **Hash Map — complement trick.**

Solution (JavaScript)

```
/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
 */
function twoSum(nums, target) {
  const seen = new Map(); // value  $\rightarrow$  index (idivaraku chusinavi)

  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i]; // ee element ki jodi enta undali

    // Aa jodi idivaraku chusi unte, jawab dorikindi
    if (seen.has(need)) {
      return [seen.get(need), i];
    }

    // Lekapotha, ee element ni future kosam gurtu pettu
    seen.set(nums[i], i);
  }

  return []; // guarantee prakaram ikkadaki raadu
}
```

Dry Run

`nums = [3,2,4]` , `target = 6` :

i	nums[i]	need = 6-nums[i]	seen.has(need)?	action
0	3	3	లేదు	seen = {3:0}
1	2	4	లేదు	seen = {3:0, 2:1}
2	4	2	అవును (index 1)	return [1, 2] ✓

గమనిక: index 0 వద్ద need=3 కూడా 3 తో సమానం, కానీ అప్పటికి map ఖాళీ — అందుకే 3+3 ని తప్పుగా తీసుకోలేదు. Insert తర్వాత check చేయడం వల్ల ఈ safety.

Complexity

- **Time: $O(n)$.** ఒక్క pass, ప్రతి step $O(1)$ map lookup.
- **Space: $O(n)$.** worst case అన్ని elements map లోకి వెళ్ళాయి.
- Brute force తో పోలిస్తే: space $O(1) \rightarrow O(n)$ ఖర్చు పెట్టి time $O(n^2) \rightarrow O(n)$ కొన్నాం.

గుర్తుంచుకోవాల్సినది

- "రెండు elements కలిపితే target?" → **complement trick** (**target** - **x** ని map లో వెతుకు), one-pass.
- **Insert తర్వాత check** కాదు — **check తర్వాత insert**. ఇది "same element రెండుసార్లు" bug ని నివారిస్తుంది.
- Array **sorted** అయితే two-pointer ($O(n)$, $O(1)$ space) కూడా option — కానీ ఇక్కడ indices కావాలి కాబట్టి sort చేస్తే original indices track చేయాలి. Unsorted + indices → hash map best.
- 3Sum, 4Sum లాంటివి ఈ complement idea పైనే కడతారు.

సాధారణ తప్పులు

- **Check ముందు insert చేయడం:** అప్పుడు `nums=[3,...]`, `target=6` లో index 0 తనతో తానే జోడి అవుతుంది → తప్పు `[0,0]`. Check-then-insert order కీలకం.
- **Values బదులు indices return చేయడం** — problem indices అడుగుతోంది. Map లో `value` → **index** ఉంచు.
- **Duplicate values:** `nums=[3,3]` — first 3 ని `{3:0}` గా పెట్టాక, second 3 వద్ద `need=3` దొరుకుతుంది → `[0,1]`. సరిగ్గా పని చేస్తుంది (map key overwrite అయినా, అవసరమైన సమయంలో lookup విజయవంతం).
- Big numbers (10^9) — JS numbers 2^{53} దాకా safe, ఇక్కడ overflow సమస్య లేదు.

7. Happy Number (LeetCode #202) — Easy

సమస్య

ఒక number `n` ఇస్తారు. అది **happy number** ఆ అని చెప్పాలి. ప్రక్రియ: number లోని digits ని square చేసి కూడు; ఆ ఫలితం మీద మళ్ళీ అదే చెయ్. ఇలా చేస్తూ పోతే **1 వద్దకి చేరితే** → **happy** (`true`). ఒకవేళ **1 కి చేరకుండా ఒక cycle లో ఇరుక్కుంటే** → **happy కాదు** (`false`).

Constraints: `1 <= n <= 2^31 - 1`.

ఉదాహరణ

```
n = 19 → true
12+92 = 82
82+22 = 68
62+82 = 100
12+02+02 = 1 → happy!
```

n = 2 → false (2 → 4 → 16 → 37 → 58 → 89 → 145 → 42 → 20 → 4 → ... cycle)

ఎలా ఆలోచించాలి

ప్రక్రియ రెండిట్లో ఒకటిగా ముగుస్తుంది: (a) 1 వద్దకి చేరుతుంది, లేదా (b) ఇంతకుముందు వచ్చిన number మళ్ళీ వస్తుంది — అంటే **cycle**. అనంతంగా కొత్త numbers రావు (గణితంగా values ఒక bounded range లో ఉంటాయి), కాబట్టి ఏదో ఒకటి తప్పక జరుగుతుంది.

అసలు సవాలు: **infinite loop ని ఎలా ఆపాలి?** — "ఈ number ఇంతకుముందు చూశానా?" అని గుర్తుంచుకుంటే, మళ్ళీ కనిపించిన క్షణం అది cycle అని తెలుస్తుంది. ఇది సరిగ్గా **seen-set** pattern.

Insight: ప్రతి కొత్త sum ని **Set** లో ఉంచు. 1 వస్తే → true. ఇప్పటికే set లో ఉన్న number మళ్ళీ వస్తే → cycle → false.

Brute Force

నిజానికి "brute force" అంటే — fixed పెద్ద సంఖ్య iterations తిప్పి, 1 రాకపోతే false అనడం. కానీ ఇది hacky, ఎన్ని iterations అన్నది arbitrary.

```
function isHappyBrute(n) {  
  for (let i = 0; i < 1000; i++) { // "chaalu" ani oohinchina limit  
    if (n === 1) return true;  
    n = sumOfSquares(n);  
  }  
  return false; // 1000 loops lo 1 raakapothe... bahusha cycle?  
}
```

- **సమస్య:** magic number **1000** — correctness guarantee లేదు, elegant కాదు. Set తో ఖచ్చితంగా (deterministically) cycle పట్టుకోవచ్చు.

Optimal Approach

Insight (Set): cycle ని seen-set తో ఖచ్చితంగా గుర్తించు.

Insight (Floyd's, O(1) space): ఇది "linked-list cycle" లాంటిదే — slow ఒక అడుగు, fast రెండు అడుగులు; అవి కలిస్తే cycle. Set అవసరం లేదు.

Plan (Set):

1. seen = new Set() .
2. n !== 1 && !seen.has(n) వరకు loop: seen.add(n) , n = sumOfSquares(n) .
3. return n === 1 .

Pattern: **Hash Set — seen-set (cycle detection).**

Solution (JavaScript)

```
/**
 * @param {number} n
 * @return {boolean}
 */
function isHappy(n) {
    const seen = new Set();

    // 1 vachhevaraku, leda okasari chusina number malli vachhevaraku
    while (n !== 1 && !seen.has(n)) {
        seen.add(n);
        n = sumOfSquares(n);
    }

    return n === 1; // 1 aithe happy; lekapothe cycle lo irukkunnam
}

// Digits ni square chesi koodu
function sumOfSquares(num) {
    let sum = 0;
    while (num > 0) {
        const digit = num % 10; // chivari digit
        sum += digit * digit;
        num = Math.floor(num / 10); // aa digit ni teeseyy
    }
    return sum;
}

// ---- Alternative: Floyd's cycle detection, O(1) space ----
function isHappyFloyd(n) {
    let slow = n;
    let fast = sumOfSquares(n);
    while (fast !== 1 && slow !== fast) {
        slow = sumOfSquares(slow); // 1 adugu
        fast = sumOfSquares(sumOfSquares(fast)); // 2 adugulu
    }
    return fast === 1;
}
```

Dry Run

n = 19 (Set version):

n	seen (ముందు)	1?	sumOfSquares(n)
19	{}	కాదు	1+81 = 82
82	{19}	కాదు	64+4 = 68
68	{19,82}	కాదు	36+64 = 100
100	{19,82,68}	కాదు	1+0+0 = 1
1	—	అవును	loop ఆగింది

return 1 === 1 → true. ✓

n = 2 : 2→4→16→37→58→89→145→42→20→4. 4 ఇప్పటికే seen లో → loop ఆగింది, n !== 1 → false. ✓

Complexity

- **Time:** $O(\log n)$ per sumOfSquares (number లో digits $\sim \log_{10} n$). మొత్తం iterations ఒక constant bound కి కుదురుతాయి (values త్వరగా చిన్నవి అవుతాయి, 243 కంటే తక్కువ range లోకి), కాబట్టి practically $O(\log n)$ మొత్తం.
- **Space:** Set version $O(\log n)$ (seen numbers). Floyd's version $O(1)$.

గుర్తుంచుకోవాల్సినది

- "ఒక ప్రక్రియ cycle లో ఇరుక్కుంటుందా / repeat అవుతుందా?" → seen-set. మళ్ళీ కనిపిస్తే cycle.
- Space $O(1)$ కావాలంటే **Floyd's tortoise-hare** — "ఇది కూడా cycle detection" అని గుర్తుపడితే linked-list technique ఇక్కడ వర్తిస్తుంది. Interview లో ఈ connection చెప్తే బాగా impress అవుతారు.
- $n \% 10$ (చివరి digit), `Math.floor(n/10)` (దాన్ని తీసేయడం) — digit manipulation యొక్క ప్రాథమిక idiom, చాలా problems లో వస్తుంది.

సాధారణ తప్పులు

- **Cycle detection లేకుండా while (n !== 1)** — 1 రాని numbers కి infinite loop. Set (లేదా Floyd's) తప్పనిసరి.
- **Math.floor మర్చిపోవడం:** $n / 10$ JS లో float ఇస్తుంది ($123/10 = 12.3$). `Math.floor` పెట్టకపోతే digit extraction తప్పు.
- **n = 1** edge case: loop condition వెంటనే false → సరిగ్గా true . చెక్ చేసుకో.
- Floyd's లో fast ని రెండు steps తప్పక కదపాలి; ఒక్క step అయితే cycle detect కాదు.

8. Contains Duplicate II (LeetCode #219) — Easy

సమస్య

Integers array nums మరియు ఒక integer k ఇస్తారు. రెండు వేర్వేరు indices i, j ఉన్నాయా — అక్కడ `nums[i] === nums[j]` మరియు `abs(i - j) <= k` — అని చెప్పాలి. అంటే: ఒకే value, k దూరంలోపు మళ్ళీ వస్తుందా?

Constraints: $1 \leq \text{nums.length} \leq 10^5$; $-10^9 \leq \text{nums}[i] \leq 10^9$; $0 \leq k \leq 10^5$.

ఉదాహరణ

```
nums = [1,2,3,1], k = 3 → true // index 0, 3: value 1, dooram 3 <= 3
nums = [1,0,1,1], k = 1 → true // index 2, 3: value 1, dooram 1 <= 1
nums = [1,2,3,1,2,3], k = 2 → false // duplicates unnayi kaani dooram > 2
```

ఎలా ఆలోచించాలి

రెండు షరతులు: (1) **value సమానం**, (2) indices **దగ్గరగా** (k లోపు). Plain "duplicate ఉందా" (Contains Duplicate I) కి ఇది distance constraint జోడించింది.

మొదటి ఆలోచన: ప్రతి value ని అది **చివరిసారి ఎక్కడ కనిపించిందో** గుర్తుంచుకో (value → **last index**). కొత్తగా అదే value వచ్చినప్పుడు, ఇప్పటి index కి last index కి తేడా $\leq k$ అయితే → true.

ఎందుకు "last index" చాలు? ఒకే value కి దగ్గరి జోడి కావాలంటే, **ఇటీవలి** occurrence తో పోల్చడమే optimal — అదే అన్నిటికంటే దగ్గర.

రెండో దారి (sliding window Set): size-k window లో duplicate ఉందా అని చూడు — window దాటిన elements ని set నుండి తీసేస్తూ.

Brute Force

అన్ని జతలు (i, j), $j - i \leq k$ లోపు, value సమానమా.

```
function containsNearbyDuplicateBrute(nums, k) {
  for (let i = 0; i < nums.length; i++) {
    for (let j = i + 1; j <= i + k && j < nums.length; j++) {
      if (nums[i] === nums[j]) return true;
    }
  }
  return false;
}
```

- **Time:** $O(n \times k)$ — ప్రతి i కి k elements ముందుకి చూస్తాం.
- **సమస్య:** n, k రెండూ 10^5 అయితే $10^{10} \rightarrow$ TLE. Hash తో $O(n)$.

Optimal Approach

Insight (last-index map): ప్రతి value యొక్క **ఇటీవలి index** ని map లో ఉంచు. అదే value మళ్ళీ వచ్చినప్పుడు $\text{distance} \leq k$ అని $O(1)$ లో check చెయ్. తర్వాత index ని update చెయ్ (కొత్తది ఇటీవలిది).

Plan:

1. `lastIndex = new Map()` (value → ఆఖరి index).
2. ప్రతి i కి: `lastIndex.has(nums[i]) && i - lastIndex.get(nums[i]) <= k → true`.
3. `lastIndex.set(nums[i], i)` (ఎప్పుడూ update).
4. Loop దాటితే → `false`.

Pattern: **Hash Map — seen-set + position tracking (sliding window)**.

Solution (JavaScript)

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {boolean}
 */
function containsNearbyDuplicate(nums, k) {
    const lastIndex = new Map(); // value -> aa value chivarisari kanipinchina index

    for (let i = 0; i < nums.length; i++) {
        // Idivaraku ee value chusam, mariyu dooram k lopu unte
        if (lastIndex.has(nums[i]) && i - lastIndex.get(nums[i]) <= k) {
            return true;
        }
        // Ee value ki chivari index ni update cheyy (recent-ade dagara)
        lastIndex.set(nums[i], i);
    }

    return false;
}

// ---- Alternative: size-k sliding window Set ----
function containsNearbyDuplicateWindow(nums, k) {
    const window = new Set();
    for (let i = 0; i < nums.length; i++) {
        if (window.has(nums[i])) return true; // window lo already unte
        window.add(nums[i]);
        if (window.size > k) window.delete(nums[i - k]); // window nunchi jarina element ni
        teeseyy
    }
    return false;
}
```

Dry Run

nums = [1,2,3,1] , k = 3 (last-index version):

i	nums[i]	has?	i - lastIndex	≤ k?	action
0	1	లేదు	—	—	set 1→0
1	2	లేదు	—	—	set 2→1
2	3	లేదు	—	—	set 3→2
3	1	అవును (0)	3-0 = 3	3 ≤ 3 ✓	return true ✓

nums = [1,2,3,1,2,3] , k = 2 : duplicates ఉన్నాయి కానీ ప్రతి జత distance 3 > 2 → అన్ని checks fail → **false**.

✓

Complexity

- **Time: $O(n)$.** ఒక్క pass, $O(1)$ map/set operations.
- **Space: $O(\min(n, k))$.** window version window size k దాకా; last-index version worst case $O(n)$ (అన్ని distinct values). Window version memory-tighter.

గుర్తుంచుకోవాల్సినది

- "Duplicate ఉందా, కానీ ఒక range/distance లోపు?" $\rightarrow$ value $\rightarrow$ **last index** map, లేదా **size-k sliding window set**.
- Sliding window + hash set combo చాలా substring/subarray problems లో (Longest Substring Without Repeating మొదలైనవి) కీలకం — ఇక్కడ దాని చిన్న రూపం.
- "ఇటీవలి occurrence మాత్రమే ముఖ్యం" అనే insight — దగ్గరి జోడీ కావాలంటే recent-తో పోల్చడమే సరిపోతుంది.

సాధారణ తప్పులు

- $<$ vs $\leq$: condition $i - j \leq k$ (equal అనుమతి). $<$ వాడితే exact-k దూరపు జతలు miss అవుతాయి.
- **Index update మర్చిపోవడం / condition లోపల మాత్రమే update చేయడం:** ప్రతి iteration లోనూ `lastIndex.set` చేయాలి (duplicate కాకపోయినా), లేదంటే positions పాతవి అవుతాయి.
- **Window version లో $i - k$ boundary:** `window.size > k` అయినప్పుడే delete; index $i - k$ negative కాకుండా చూసుకోవడం (size condition ఇది guarantee చేస్తుంది).
- **$k = 0$ edge case:** ఏ రెండు distinct indices దూరం 0 అవ్వదు $\rightarrow$ ఎప్పుడూ false. Code సహజంగా handle చేస్తుంది ($i - \text{last} \leq 0$ కేవలం same index కి, అది జరగదు).

9. Longest Consecutive Sequence (LeetCode #128) — Medium

సమస్య

Unsorted integers array `nums` ఇస్తారు. అందులో ఉన్న elements తో ఏర్పడే **పొడవైన consecutive (వరుస) sequence** యొక్క length కనుక్కో. ఉదా. `[100, 4, 200, 1, 3, 2]` లో `1,2,3,4` — length 4. **షరతు:** algorithm $O(n)$ time లో run అవ్వాలి.

Constraints: $0 \leq \text{nums.length} \leq 10^5$; $-10^9 \leq \text{nums}[i] \leq 10^9$.

ఉదాహరణ

```
nums = [100,4,200,1,3,2]    → 4    // [1,2,3,4]
nums = [0,3,7,2,5,8,4,6,0,1] → 9    // [0..8]
nums = []                  → 0
```

ఎలా ఆలోచించాలి

మొదట మనసులో వచ్చే idea: **sort చేసి**, పక్క పక్క వరుస numbers ని లెక్కించడం. అది $O(n \log n)$ – పని చేస్తుంది కానీ problem **$O(n)$** అడుగుతోంది. Sorting అనుమతి లేదు (spirit ప్రకారం).

$O(n)$ కావాలంటే **hash set** ఆలోచించు – $O(1)$ lookups. అన్ని numbers ని set లో పెడితే, "ఈ number ఉందా?" క్షణంలో తెలుస్తుంది.

కీలక **insight**: ప్రతి number నుండి sequence ని కుడివైపు ($num+1, num+2, \dots$) పెంచుతూ లెక్కించవచ్చు. కానీ ప్రతి number నుండి మొదలుపెడితే పని మళ్ళీ మళ్ళీ ($O(n^2)$). దీన్ని ఆపడానికి – **sequence యొక్క మొదటి number నుండి మాత్రమే** లెక్కించు. ఒక number **num** sequence start అవ్వాలంటే, $num - 1$ set లో **ఉండకూడదు** (లేకపోతే అది start కాదు, మధ్యలో ఉంది). ఈ ఒక్క check తో మొత్తం $O(n)$ అవుతుంది – ప్రతి element inner-loop లో గరిష్టంగా ఒక్కసారి visit అవుతుంది.

Brute Force

ప్రతి number **num** కి, $num+1, num+2, \dots$ array లో ఉన్నాయా అని linear search చేస్తూ sequence పెంచడం.

```
function longestConsecutiveBrute(nums) {
  let longest = 0;
  for (const num of nums) {
    let current = num, length = 1;
    // num+1, num+2... array lo unnaya ani prati sari vetaku (O(n))
    while (nums.includes(current + 1)) { current++; length++; }
    longest = Math.max(longest, length);
  }
  return longest;
}
```

- **Time:** $O(n^3)$ worst case (**includes** $O(n)$ × while $O(n)$ × outer $O(n)$). Sort version $O(n \log n)$.
- **సమస్య:** $O(n)$ requirement ని ఉల్లంఘిస్తుంది.

Optimal Approach

Insight: అన్నిటినీ **Set** లో పెట్టు ($O(1)$ lookup, duplicates పోతాయి). ప్రతి number కి – అది **sequence start** అయితే మాత్రమే (అంటే $num-1$ set లో లేకపోతే) – కుడివైపు ఎంత పొడవు పోతుందో లెక్కించు.

ఎందుకు $O(n)$? inner **while** loop మొత్తం program లో కలిపి గరిష్టంగా n సార్లు మాత్రమే run అవుతుంది – ఎందుకంటే అది sequence starts నుండే మొదలై, ప్రతి element ని ఒక్కసారి చాటుతుంది. అంటే outer + inner కలిపి $O(n)$.

Plan:

1. **set** = new **Set**(nums) .
2. ప్రతి **num** in set కి: **set.has(num - 1)** అయితే skip (start కాదు).
3. లేదంటే start: **current** = num , **length** = 1 ; **set.has(current + 1)** వరకు **current++** , **length++** .
4. **longest** = **max**(longest, **length**) .

Pattern: **Hash Set — $O(1)$ lookup + "count only from sequence starts."**

Solution (JavaScript)

```
/**
 * @param {number[]} nums
 * @return {number}
 */
function longestConsecutive(nums) {
  const set = new Set(nums); // O(1) lookup, duplicates poyayi
  let longest = 0;

  for (const num of set) {
    // 'num' oka sequence START aithe matrame lekkinchu:
    // num-1 set lo lekapothe, num modati element
    if (!set.has(num - 1)) {
      let current = num;
      let length = 1;

      // Kudivaipu varusaga entha poddo veatuku
      while (set.has(current + 1)) {
        current++;
        length++;
      }

      longest = Math.max(longest, length);
    }
  }

  return longest;
}
```

Dry Run

nums = [100, 4, 200, 1, 3, 2] → set = {100,4,200,1,3,2} :

num	num-1 in set?	start?	sequence గణన	length
100	99? లేదు	అవును	101? లేదు	1
4	3? అవును	కాదు	skip	—
200	199? లేదు	అవును	201? లేదు	1
1	0? లేదు	అవును	2✓→3✓→4✓→5? లేదు	4
3	2? అవును	కాదు	skip	—
2	1? అవును	కాదు	skip	—

longest = 4. ✓ గమనిక: sequence 1,2,3,4 కేవలం num=1 (అసలు start) వద్దే లెక్కించబడింది – 2,3,4 skip అయ్యాయి. అందుకే O(n).

Complexity

- **Time: $O(n)$.** Set build $O(n)$. Outer loop n elements, కానీ inner **while** మొత్తం కలిపి $O(n)$ (ప్రతి element ఒక్కసారే inner-visit). మొత్తం $O(n)$.
- **Space: $O(n)$.** Set లో n elements.

గుర్తుంచుకోవాల్సినది

- " **$O(n)$ లో sorted-లాంటి పని (consecutive/ordering) కావాలి, కానీ sort చేయకూడదు**" → **hash set** + smart iteration.
- "**పని మొదలుపెట్టడానికి సరైన starting point ఎంచుకో**" – ఇక్కడ sequence starts ($num-1$ లేని numbers) నుండి మొదలుపెట్టడం $O(n^2)$ ని $O(n)$ చేసిన మంత్రం. ఈ "start from the boundary" idea చాలా చోట్ల ఉపయోగపడుతుంది.
- Set iteration (**for...of set**) duplicates ని దాటవేస్తుంది – free deduplication.

సాధారణ తప్పులు

- **Start check లేకుండా ప్రతి number నుండి లెక్కించడం** → $O(n^2)/O(n^3)$, requirement violate. `!set.has(num-1)` కీలకం.
- **nums array మీద iterate చేసి duplicates మళ్ళీ process చేయడం** – set మీద iterate చేస్తే duplicates ఒక్కసారే. (Array మీద చేసినా correctness ఉంటుంది కానీ set cleaner.)
- **Empty array:** longest 0 తో initialize చెయ్య – `[]` కి సరిగ్గా 0 return అవుతుంది.
- **Sort వాడాలనే టెంప్లేషన్:** పని చేస్తుంది ($O(n \log n)$) కానీ interviewer $O(n)$ అడిగితే fail. Hash set approach తెలిసి ఉండాలి.

Pattern: Intervals

వివరణ

Interval అంటే ఒక `[start, end]` జత – ఒక range ని సూచిస్తుంది. ఉదా. ఒక meeting `[9, 10]` (9 గంటల నుండి 10 దాకా), ఒక booking `[2, 5]` . Interval problems లో మనకి ఇలాంటి ranges యొక్క list ఇస్తారు, వాటిని **merge చెయ్య, overlaps కనుక్కో, insert చెయ్య, లేదా minimum ఏదో లెక్కించు** అని అడుగుతారు.

ఈ pattern యొక్క **ఏకైక అతిపెద్ద రహస్యం:** దాదాపు అన్ని interval problems మొదట **sort చేయడం** తో మొదలవుతాయి – **start** బట్టి లేదా **end** బట్టి. Sort చేశాక, ranges ఒక క్రమంలో వస్తాయి, అప్పుడు ఒక్క pass లో పక్క పక్క intervals ని పోల్చుకుంటూ పని పూర్తి చేయవచ్చు.

Overlap అంటే ఏమిటి? రెండు intervals $A = [a1, a2]$ మరియు $B = [b1, b2]$ (A ముందు మొదలైతే, $a1 \leq b1$) – ఇవి overlap అవుతాయంటే **$b1 \leq a2$** (B start, A end లోపు మొదలైంది). అంటే ఒకదాని start మరొకదాని end ని దాటకముందే వస్తే – అవి కలుస్తాయి.

ఉపయోగం	ఏం చేస్తుంది	ఉదాహరణ problems
Merge / overlap	కలిసే ranges ని ఒకటిగా చెయ్	Merge Intervals, Insert Interval
Sort by start	క్రమంగా process చేయడానికి	Merge, Insert
Sort by end + greedy	గరిష్టంగా ఎక్కువ non-overlapping / కనిష్ట points	Burst Balloons, Non-overlapping
Sweep line	events (start/end) ని timeline లో process	Meeting Rooms II
Grouping ranges	వరుస numbers ని range గా	Summary Ranges

Real-life Scenario

Intervals = ఒక calendar లోని meetings. నీ రోజులో [9–10], [9:30–11], [14–15] meetings ఉన్నాయనుకో. "నాకు ఎన్ని distinct busy blocks ఉన్నాయి?" అని తెలుసుకోవాలంటే – మొదట meetings ని **start time బట్టి వరుసగా పేర్చు** (sort). తర్వాత పైనుండి కిందకి చూస్తూ, ఒక meeting ముగియకముందే తర్వాతిది మొదలైతే వాటిని **ఒకే busy block గా కలుపు** (merge). [9–10] మరియు [9:30–11] overlap → [9–11]. ఇదే merge intervals!

గజిబిజి meetings ని ఒక క్రమంలో పేర్చుకుండా merge చేయడం అసాధ్యం – అందుకే **sort మొదటి అడుగు**.

ఎలా గుర్తించాలి (Recognition Signals)

- Input [**start**, **end**] జతల array గా ఉంది → interval problem.
- "**merge / combine / overlap / intersect**" అనే మాటలు → sort by start + merge.
- "**minimum number of** (rooms / arrows / platforms) to cover/handle all" → sort + **greedy** (తరచుగా sort by end).
- "**maximum non-overlapping intervals**" → sort by end + greedy.
- "ranges ని **insert / add** చెయ్" → sorted list లో మూడు భాగాలు (before / overlap / after).
- వరుస integers ని **compact ranges** గా చూపు → Summary Ranges.

Template Code (JavaScript)

```
// ---- 1. Sort by start (merge problems ki) ----
intervals.sort((a, b) => a[0] - b[0]);

// ---- 2. Sort by end (greedy problems ki) ----
intervals.sort((a, b) => a[1] - b[1]);

// ---- 3. Merge overlapping intervals (skeleton) ----
function mergeSkeleton(intervals) {
  intervals.sort((a, b) => a[0] - b[0]); // start batti sort
  const result = [];
  for (const [start, end] of intervals) {
    const last = result[result.length - 1];
    // last unte, mariyu ippati start last end ni daatakapothe → overlap → merge
    if (last && start <= last[1]) {
```

```

    last[1] = Math.max(last[1], end); // end ni podiginchu
  } else {
    result.push([start, end]);      // kotta block
  }
}
return result;
}

// ---- 4. Overlap check (two intervals) ----
function overlaps(a, b) {
  return a[0] <= b[1] && b[0] <= a[1]; // okadi start marokadi end ni daataledu
}

```

Comparator warning: JS లో `arr.sort()` (comparator లేకుండా) elements ని **strings** గా పోల్చుతుంది — `[10, 2]` ని `["10", "2"]` గా చూసి తప్పు order ఇస్తుంది. Intervals కి ఎప్పుడూ `sort((a, b) => a[0] - b[0])` లాంటి numeric comparator ఇవ్వు.

Complexity

- **Sort: $O(n \log n)$** — ఇదే చాలా interval solutions లో dominant cost.
- **Merge/scan pass: $O(n)$** — sorted list మీద ఒక్క pass.
- **మొత్తం:** సాధారణంగా **$O(n \log n)$ time.**
- **Space:** result కి $O(n)$; sort in-place అయితే extra $O(\log n)$ (recursion stack) — implementation బట్టి.

10. Summary Ranges (LeetCode #228) — Easy

సమస్య

Sorted, duplicates లేని integers array `nums` ఇస్తారు. దానిని **ranges** యొక్క smallest sorted list గా సూచించాలి. వరుస numbers ని `"a->b"` గా, ఒంటరి numbers ని `"a"` గా రాయాలి. ప్రతి number ఖచ్చితంగా ఒక range లో ఉండాలి.

Constraints: `0 <= nums.length <= 20`; `-231 <= nums[i] <= 231 - 1`; array sorted & unique.

ఉదాహరణ

```

nums = [0,1,2,4,5,7]    → ["0->2", "4->5", "7"]
nums = [0,2,3,4,6,8,9]  → ["0", "2->4", "6", "8->9"]

```

ఎలా ఆలోచించాలి

Array ఇప్పటికే **sorted & unique** — అంటే వరుస numbers (`x`, `x+1`, `x+2...`) పక్క పక్కనే ఉంటాయి. మనం చేయాల్సింది: ఒక range start గుర్తుంచుకొని, వరుస కొనసాగినంత వరకు ముందుకి వెళ్లి, వరుస తెగినప్పుడు (`nums[i+1] !== nums[i] + 1`) ఆ range ని close చెయ్.

Insight: ఇది classic **grouping consecutive elements** — ఒక్క pass, two-pointer లాంటి scan. Hashing అవసరం లేదు; sorted property చాలు. (ఇది intervals section లో ఉంది ఎందుకంటే output కూడా ranges/intervals రూపంలో ఉంటుంది.)

Brute Force

నిజానికి ఇక్కడ "brute force" అంటే — ప్రతి number కి తర్వాతి numbers అన్నీ scan చేసి range extend చేయడం, కానీ అది కూడా చివరికి linear scan అవుతుంది. అసలైన naive తప్పు: **runs ని గుర్తించకుండా** ప్రతి number ని విడిగా range గా రాయడం (["0", "1", "2", ...]) — ఇది "smallest list" కాదు.

```
// Tappu naive: prati number viddiga (smallest list kaadu)
function summaryRangesNaive(nums) {
  return nums.map(String); // ["0", "1", "2", "4", "5", "7"] — merge cheyaledu
}
```

- **సమస్య:** requirement "smallest sorted list" ఉల్లంఘన. వరుస runs ని merge చేయాలి.

Optimal Approach

Insight: Two-pointer scan — **start** వద్ద ఒక range మొదలవుతుంది; వరుస కొనసాగినంత వరకు **i** ముందుకి; తెగినప్పుడు `[start, nums[i]]` ని format చేసి push.

Plan:

1. `i = 0` ; array end దాకా:
2. `start = nums[i]` . `i+1 < n && nums[i+1] === nums[i] + 1` వరకు `i++` .
3. `end = nums[i]` . `start === end` అయితే `"start"` , లేదంటే `"start->end"` .
4. `i++` (తర్వాతి range).

Pattern: **Intervals / grouping consecutive — single pass.**

Solution (JavaScript)

```
/**
 * @param {number[]} nums
 * @return {string[]}
 */
function summaryRanges(nums) {
  const result = [];
  const n = nums.length;
  let i = 0;

  while (i < n) {
    const start = nums[i];

    // Varusa konasaginanta varaku i ni munduki tosuku
    while (i + 1 < n && nums[i + 1] === nums[i] + 1) {
      i++;
    }
  }
}
```

```

const end = nums[i];
// Okkate number aithe "a", lekapothe "a->b"
result.push(start === end ? `${start}` : `${start}->${end}`);

i++; // tarvati range modalu
}

return result;
}

```

Dry Run

nums = [0,1,2,4,5,7] :

i (start వద్ద)	start	inner while ముగిశాక i	end	push
0	0	2 (0→1→2 వరుస)	2	"0->2"
3	4	4 (4→5 వరుస)	5	"4->5"
5	7	5 (7 ఒంటరి)	7	"7"

result = ["0->2", "4->5", "7"] .✓

Complexity

- **Time: O(n).** ప్రతి element inner/outer కలిపి ఒక్కసారే visit అవుతుంది.
- **Space: O(1)** extra (output మినహా). Output కి O(number of ranges).

గుర్తుంచుకోవాల్సినది

- **"Sorted array లో వరుస runs ని group చెయ్"** → two-pointer scan, run తెగే చోట close చెయ్.
- Array sorted & unique అయినప్పుడు hashing అవసరం లేదు — **ordering ని exploit చెయ్**. ఇది interval/scan problems యొక్క DNA.
- **start === end ? single : range** — ఒంటరి vs range format ని ఒక్క ternary తో handle చెయ్.

సాధారణ తప్పులు

- **i + 1 < n bounds check మర్చిపోవడం** → nums[i+1] array దాటి **undefined** → wrong comparison. Inner loop లో boundary తప్పనిసరి.
- **Inner loop తర్వాత i++ మర్చిపోవడం** → infinite loop (i కదలదు).
- **ఒంటరి number ని -> తో format చేయడం ("7->7")** — start === end check అవసరం.
- **Overflow:** JS numbers doubles కాబట్టి nums[i]+1 2³¹-1 వద్ద కూడా safe (2⁵³ దాకా). C++/Java లో ఇది overflow trap.

11. Merge Intervals (LeetCode #56) – Medium

సమస్య

Intervals యొక్క array `intervals` ఇస్తారు, ప్రతిది `[start, end]`. **Overlap** అయ్యే అన్ని intervals ని merge చేసి, non-overlapping intervals యొక్క array return చేయ్ (input లోని అన్ని ranges ని cover చేస్తూ).

Constraints: `1 <= intervals.length <= 10^4`; `intervals[i] = [start_i, end_i]`; `0 <= start_i <= end_i <= 10^4`.

ఉదాహరణ

```
intervals = [[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]]
// [1,3] & [2,6] overlap (2 <= 3) → [1,6]
intervals = [[1,4],[4,5]] → [[1,5]] // 4 <= 4, touch kuda merge
```

ఎలా ఆలోచించాలి

Overlap చేసేవాటిని కలపాలి. కానీ input గజిబిజిగా (unsorted) ఉంటే, ఒక interval మరొకదానితో overlap అవుతుందే లేదో తెలుసుకోవడం కష్టం – ఏదైనా దేనితోనైనా overlap అవ్వచ్చు.

కీలక insight: ముందు **start time బట్టి sort** చేయ్. అప్పుడు overlap చేసేవి **తప్పకుండా పక్క పక్కనే** వస్తాయి. ఇక ఒక్క pass చాలు: `result` లోని చివరి interval తో ప్రస్తుత interval overlap అవుతోందా (`current.start <= last.end`) అని చూడు – అయితే `last.end` ని పొడిగించు (merge), లేదంటే కొత్త interval గా జోడించు.

Sort లేకుండా ఇది $O(n^2)$ (ప్రతి జతని పోల్చాలి); sort తో $O(n \log n)$.

Brute Force

Sort లేకుండా – ఏదైనా రెండు intervals overlap అయితే merge చేస్తూ, మార్పు ఆగేదాకా repeat.

```
function mergeBrute(intervals) {
  let merged = intervals.map(x => [...x]);
  let changed = true;
  while (changed) {
    changed = false;
    outer:
    for (let i = 0; i < merged.length; i++) {
      for (let j = i + 1; j < merged.length; j++) {
        // overlap aithe kalipi, j ni teeseeyy
        if (merged[i][0] <= merged[j][1] && merged[j][0] <= merged[i][1]) {
          merged[i] = [Math.min(merged[i][0], merged[j][0]),
                       Math.max(merged[i][1], merged[j][1])];
          merged.splice(j, 1);
          changed = true;
          break outer;
        }
      }
    }
  }
}
```

```
    return merged;
}
```

- **Time:** $O(n^3)$ worst case (repeated passes × nested compare × splice).
- **సమస్య:** చాలా నెమ్మది. Sort చేస్తే overlaps పక్క పక్కనే వచ్చి $O(n \log n)$.

Optimal Approach

Insight: Sort by start → overlaps adjacent → ఒక్క pass merge.

Plan:

1. `intervals.sort((a,b) => a[0]-b[0])` .
2. `result = []` . ప్రతి `[start,end]` కి:
 - `last = result[last]` . `last && start <= last[1] → last[1] = max(last[1], end)` (merge).
 - లేదంటే `result.push([start,end])` .
3. `result` return.

Pattern: **Intervals — sort by start + merge.**

Solution (JavaScript)

```
/**
 * @param {number[][]} intervals
 * @return {number[][]}
 */
function merge(intervals) {
    // Step 1: start time batti sort – overlaps pakka pakkane vastayi
    intervals.sort((a, b) => a[0] - b[0]);

    const result = [];

    for (const [start, end] of intervals) {
        const last = result[result.length - 1];

        // last unte, mariyu ippati start last end ni daatakapothe → overlap
        if (last && start <= last[1]) {
            last[1] = Math.max(last[1], end); // end ni podiginchu (merge)
        } else {
            result.push([start, end]); // overlap ledu → kotta interval
        }
    }

    return result;
}
```

Dry Run

`intervals = [[1,3],[2,6],[8,10],[15,18]]` (ఇప్పటికే start-sorted):

current	last	start <= last[1] ?	action	result
[1,3]	—	—	push	[[1,3]]
[2,6]	[1,3]	$2 \leq 3$ ✓	merge: end=max(3,6)=6	[[1,6]]
[8,10]	[1,6]	$8 \leq 6$? లేదు	push	[[1,6], [8,10]]
[15,18]	[8,10]	$15 \leq 10$? లేదు	push	[[1,6], [8,10], [15,18]]

Output: [[1,6], [8,10], [15,18]] . ✓

Complexity

- **Time: $O(n \log n)$.** Sort dominates; merge pass $O(n)$.
- **Space: $O(n)$** result కి (లేదా sort/output మినహాయస్తే $O(\log n)$ auxiliary).

గుర్తుంచుకోవాల్సినది

- **Interval problems యొక్క రాజు.** "merge / overlap / combine ranges" వింటే → **sort by start, అపై start <= last.end అయితే last.end = max(...)** .
- **Merge లో max తప్పనిసరి:** [1,6] తర్వాత [2,4] వస్తే (fully inside), end ని 4 కి కుదించకూడదు — $\max(6,4)=6$. ఇది తరచుగా bug.
- **<= (touch కూడా merge):** [1,4] & [4,5] → [1,5] . Problem "touching = overlap" అనుకుంటుంటే లేదే గమనించు (ఇది అనుకుంటుంది).
- Insert Interval, Non-overlapping Intervals, Meeting Rooms — అన్నీ ఈ skeleton యొక్క variations.

సాధారణ తప్పులు

- **Sort మర్చిపోవడం లేదా sort() comparator లేకుండా** — lexicographic sort వల్ల [[1,3], [12,15], [2,6]] తప్పు order → wrong merge. ఎప్పుడూ `sort((a,b)=>a[0]-b[0])` .
- **Math.max బదులు నేరుగా end assign చేయడం** — nested interval end ని కుదించేస్తుంది (bug పైన చెప్పినట్టు).
- **start < last[1] (strict)** వాడితే touching intervals ([1,4], [4,5]) merge అవ్వవు. <= వాడు.
- **Empty result లో last undefined** — last && guard తో handle. లేకపోతే `undefined[1]` crash.

12. Insert Interval (LeetCode #57) — Medium

సమస్య

ఇప్పటికే **sorted (by start), non-overlapping** intervals యొక్క array `intervals` మరియు ఒక కొత్త `newInterval` ఇస్తారు. `newInterval` ని సరైన చోట insert చేసి, ఇంకా అవసరమైతే merge చేసి, తిరిగి sorted non-overlapping array return చేయ్.

Constraints: $0 \leq \text{intervals.length} \leq 10^4$; intervals sorted by start, non-overlapping; `newInterval = [start, end]`.

ఉదాహరణ

```
intervals = [[1,3],[6,9]], newInterval = [2,5]      → [[1,5],[6,9]]
// [2,5] overlaps [1,3] → [1,5]; [6,9] alone
intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
→ [[1,2],[3,10],[12,16]] // [4,8] merges [3,5],[6,7],[8,10]
```

ఎలా ఆలోచించాలి

Input **ఇప్పటికే sorted & non-overlapping** – ఇది పెద్ద కానుక. మనం మళ్ళీ sort చేయనవసరం లేదు (అది $O(n \log n)$). ఒక్క linear pass ($O(n)$) చాలు.

కొత్త interval ని బట్టి intervals ని **మూడు భాగాలు** గా విడగొట్టు:

1. **ఎడమ (left):** newInterval మొదలవ్వకముందే పూర్తిగా ముగిసిన intervals (`interval.end < newInterval.start`) – వీటిని అలాగే copy చేయ్.
2. **మధ్య (overlap):** newInterval తో **తాకే/overlap** అయ్యే intervals (`interval.start <= newInterval.end`) – వీటన్నిటినీ newInterval లోకి merge చేయ్ (start ని min, end ని max).
3. **కుడి (right):** newInterval ముగిశాక మొదలయ్యే intervals – వీటిని అలాగే copy చేయ్.

Merged newInterval ని left తర్వాత, right ముందు పెట్టు.

Brute Force

newInterval ని array లో పెట్టి, మొత్తం Merge Intervals (#56) algorithm run చేయడం.

```
function insertBrute(intervals, newInterval) {
  const all = [...intervals, newInterval];
  all.sort((a, b) => a[0] - b[0]); // malli sort (O(n log n))
  const result = [];
  for (const [s, e] of all) {
    const last = result[result.length - 1];
    if (last && s <= last[1]) last[1] = Math.max(last[1], e);
    else result.push([s, e]);
  }
  return result;
}
```

- **Time:** $O(n \log n)$ – sorted property ని వృథా చేసి మళ్ళీ sort.
- **సమస్య:** పని చేస్తుంది కానీ optimal కాదు. Already-sorted ని exploit చేస్తే $O(n)$.

Optimal Approach

Insight: Sorted కాబట్టి మూడు భాగాలను ఒక్క pass లో దాటవచ్చు – sort అవసరం లేదు.

Plan:

1. `result = []` , `i = 0` .
2. **Left:** `intervals[i][1] < newInterval[0]` వరకు – push, `i++` .
3. **Overlap:** `intervals[i][0] <= newInterval[1]` వరకు – `newInterval[0] = min(...)` , `newInterval[1] = max(...)` , `i++` . తర్వాత `result.push(newInterval)` .
4. **Right:** మిగిలినవి – push.

Pattern: **Intervals – three-phase linear merge (sorted exploit).**

Solution (JavaScript)

```
/**
 * @param {number[][]} intervals
 * @param {number[]} newInterval
 * @return {number[][]}
 */
function insert(intervals, newInterval) {
    const result = [];
    const n = intervals.length;
    let i = 0;

    // Phase 1 – Edama: newInterval modalavvakamunde mugise intervals
    while (i < n && intervals[i][1] < newInterval[0]) {
        result.push(intervals[i]);
        i++;
    }

    // Phase 2 – Madhya: overlap ayye anniti ni newInterval loki merge
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    result.push(newInterval); // merge aina (leda ala unna) newInterval

    // Phase 3 – Kudi: newInterval mugisaka modalayye intervals
    while (i < n) {
        result.push(intervals[i]);
        i++;
    }

    return result;
}
```

Dry Run

`intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]]` , `newInterval = [4,8]` :

Phase 1 (left): `intervals[0]=[1,2]` , `2 < 4` ✓ → push `[1,2]` , `i=1`. `intervals[1]=[3,5]` , `5 < 4` ? లేదు
→ ఆగు.

Phase 2 (overlap): newInterval = [4,8].

- [3,5] : 3 <= 8 ✓ → newInterval = [min(4,3), max(8,5)] = [3,8] , i=2.
- [6,7] : 6 <= 8 ✓ → [3,8] (max(8,7)=8), i=3.
- [8,10] : 8 <= 8 ✓ → [3, max(8,10)] = [3,10] , i=4.
- [12,16] : 12 <= 10 ? లేదు → ఆగు. push [3,10] .

Phase 3 (right): [12,16] → push. i=5.

result = [[1,2], [3,10], [12,16]] . ✓

Complexity

- **Time: O(n).** ప్రతి interval ఒక్క phase లో ఒక్కసారే touch. Sort లేదు.
- **Space: O(n)** result కి.

గుర్తుంచుకోవాల్సినది

- "Sorted intervals లో ఒకటి insert చెయ్" → **మూడు phases: left (before) → merge overlaps → right (after).** Sort అవసరం లేదు – ఇదే optimality మంత్రం.
- **Left condition** end < newStart (strict <) vs **overlap condition** start <= newEnd (<=) – ఈ boundary లు జాగ్రత్తగా. Touching (end == newStart) ని overlap గా treat చేయాలంటే left లో strict < వాడు.
- newInterval ని **in-place mutate** చేస్తూ overlaps ని absorb చేయడం – clean idiom. కావాలంటే copy తీసుకో.
- ఇది Merge Intervals యొక్క "one new element" special case – sorted input వల్ల చౌక.

సాధారణ తప్పులు

- **Boundary strict/non-strict తారుమారు:** left లో <= వాడితే touching interval ని తప్పుగా separate గా ఉంచుతుంది. Left = end < newStart , overlap = start <= newEnd .
- **Merge తర్వాత** result.push(newInterval) **మర్చిపోవడం** – కొత్త interval మాయమవుతుంది.
- **Math.min / Math.max** లో newInterval ని **update చేయకుండా** intervals[i] ని వాడటం – merged range తప్పు.
- **Empty intervals** – అన్ని phases skip అయి కేవలం newInterval push అవుతుంది → [[newInterval]] . సరిగ్గా పని చేస్తుంది, verify చేసుకో.
- **i** ని ప్రతి while లో **increment చేయకపోవడం** → infinite loop.

13. Minimum Number of Arrows to Burst Balloons (LeetCode #452) – Medium

సమస్య

Balloons యొక్క array points ఇస్తారు, ప్రతిది [x_start, x_end] – ఆ balloon x-axis మీద ఆ range లో వ్యాపించి ఉంది. ఒక **arrow** ని ఏదైనా x coordinate వద్ద నిటారుగా (vertically) పైకి వదిలితే, ఆ x ని cover చేసే అన్ని balloons

పగులుతాయి ($x_{start} \leq x \leq x_{end}$). అన్ని balloons పగలడానికి కావాల్సిన కనిష్ట arrows ఎన్నో చెప్పు.

Constraints: $1 \leq \text{points.length} \leq 10^5$; $\text{points}[i] = [x_{start}, x_{end}]$; $-2^{31} \leq x_{start} \leq x_{end} \leq 2^{31} - 1$.

ఉదాహరణ

```
points = [[10,16],[2,8],[1,6],[7,12]] → 2
// arrow x=6: [2,8],[1,6] pagulutayi; arrow x=11: [10,16],[7,12]
points = [[1,2],[3,4],[5,6],[7,8]] → 4 // edi kooda overlap kaadu
points = [[1,2],[2,3],[3,4],[4,5]] → 2 // x=2: [1,2],[2,3]; x=4: [3,4],[4,5]
```

ఎలా ఆలోచించాలి

ఒక arrow గరిష్టంగా ఎక్కువ balloons ని పగలగొట్టాలంటే, ఒకదానితో ఒకటి overlap అయ్యే balloons ని ఒకే arrow తో కొట్టాలి. కాబట్టి ప్రశ్న నిజానికి: "ఎన్ని groups of overlapping balloons ఉన్నాయి?" – ప్రతి overlapping group కి ఒక arrow.

ఇది **greedy interval** problem. కీలక నిర్ణయం: **ఏ బట్టి sort చేయాలి? – end (x_{end}) బట్టి.** ఎందుకు? ఒక arrow ని ఎప్పుడూ **ప్రస్తుత group లోని అతి తక్కువ end** వద్ద వదిలితే, అది ఆ balloon ని పగలగొడుతూనే, తర్వాత వచ్చే వీలైనన్ని balloons ని కూడా cover చేసే **గరిష్ట అవకాశం** ఇస్తుంది. End వద్ద కొట్టడం అనేది "as far right as possible while still bursting this balloon" – greedy లో classic.

Insight: Sort by end. మొదటి balloon end వద్ద arrow పెట్టు. తర్వాత balloon **start** ఆ arrow position ని **దాటితే** ($\text{start} > \text{arrowPos}$), అది కొత్త group → కొత్త arrow (దాని end వద్ద). లేదంటే ప్రస్తుత arrow దాన్ని కూడా పగలగొడుతుంది.

Brute Force

అన్ని overlapping combinations ని పరిశీలించడం (ఏ balloons ఒకే arrow తో పోతాయో అన్ని రకాలుగా try) – ఇది exponential, అసాధ్యం. లేదా interval-graph లో "minimum points to stab all intervals" ని నిర్మించడం – అది కూడా sort లేకుండా $O(n^2)$.

```
// Naive alochana: prati balloon ki, migatha vati tho overlap check ( $O(n^2)$ ),
// group cheyadaniki – kaani greedy correctness kosam sort avasaram.
// Sort lekunda "minimum stabbing points" ni serigga rabbattadam kashtam.
```

- సమస్య: greedy + sort లేకుండా correctness guarantee ఇవ్వడం కష్టం; naive approaches $O(n^2)$ లేదా అంతకంటే ఎక్కువ. Sort-by-end greedy → $O(n \log n)$, నిరూపితంగా optimal.

Optimal Approach

Insight: Sort by end (x_{end}). Greedy: ప్రతి group కి, అతి-ఎడమ end వద్ద ఒక arrow; తర్వాతి balloon start ఆ arrow ని దాటితే కొత్త arrow.

ఎందుకు end బట్టి, start బట్టి కాదు? End వద్ద కొట్టడం – ప్రస్తుత balloon తప్పకుండా పోతుంది, కానీ arrow సాధ్యమైనంత కుడివైపు ఉంటుంది కాబట్టి తర్వాత overlapping balloons ఎక్కువ cover అవుతాయి. Start బట్టి sort చేస్తే ఈ greedy సూటిగా రాదు.

Plan:

1. `points.sort((a,b) => a[1]-b[1])` (end బట్టి).
2. `arrows = 1` , `arrowPos = points[0][1]` (మొదటి end).
3. ప్రతి తర్వాతి balloon కి: `start > arrowPos` అయితే → కొత్త arrow: `arrows++` , `arrowPos = points[i][1]` .
4. `arrows` return.

Pattern: **Intervals – sort by end + greedy.**

Solution (JavaScript)

```
/**
 * @param {number[][]} points
 * @return {number}
 */
function findMinArrowShots(points) {
    if (points.length === 0) return 0;

    // End (x_end) batti sort – greedy correctness kosam idi kelakam
    points.sort((a, b) => a[1] - b[1]);

    let arrows = 1; // modati balloon ki okka arrow
    let arrowPos = points[0][1]; // aa arrow ni modati balloon END vaddaki pettu

    for (let i = 1; i < points.length; i++) {
        // Ee balloon start, present arrow ni daatithe → idi kotta group
        if (points[i][0] > arrowPos) {
            arrows++; // kotta arrow avasaram
            arrowPos = points[i][1]; // dani END vaddaki pettu
        }
        // Lekapothe present arrow ee balloon ni kooda pagalagoduthundi – em cheyavaddu
    }

    return arrows;
}
```

Comparator note: `a[1] - b[1]` పెద్ద coordinates ($\pm 2^{31}$) కి JS లో safe – numbers doubles (2^{63} దాకా), overflow లేదు. (C++/Java లో `a[1]-b[1]` int overflow అవుతుంది; అక్కడ `Integer.compare` వాడాలి.)

Dry Run

`points = [[10,16],[2,8],[1,6],[7,12]] :`

Sort by end: `[[1,6],[2,8],[7,12],[10,16]]` (ends 6,8,12,16).

- `arrows=1` , `arrowPos = 6` (first balloon end).

- $[2,8]$: $\text{start}=2 > 6$? లేదు $\rightarrow$ arrow $x=6$ దీన్ని కూడా పగలగొడుతుంది ($2 \leq 6 \leq 8$). ఏమీ చేయవద్దు.
- $[7,12]$: $\text{start}=7 > 6$? **అవును** $\rightarrow$ కొత్త arrow: $\text{arrows}=2$, $\text{arrowPos}=12$.
- $[10,16]$: $\text{start}=10 > 12$? లేదు $\rightarrow$ arrow $x=12$ దీన్ని పగలగొడుతుంది ($10 \leq 12 \leq 16$).

return 2 . ✓ (arrow $x=6 \rightarrow \{[1,6],[2,8]\}$, arrow $x=12 \rightarrow \{[7,12],[10,16]\}$.)

Complexity

- **Time: $O(n \log n)$** . Sort dominates; greedy pass $O(n)$.
- **Space: $O(1)$** (in-place sort మినహా; sort auxiliary $O(\log n)$).

గుర్తుంచుకోవాల్సినది

- **"Minimum arrows/points/resources to cover/stab all intervals" $\rightarrow$ sort by END + greedy.** ప్రతి కొత్త group కి ఒక resource.
- **Sort key ఎంపిక = problem పరిష్కారానికి కీలకం.** Merge = sort by start; "min points to stab" / "max non-overlapping" = sort by **end**. ఈ తేడా interview లో బాగా అడుగుతారు.
- ఇది "Non-overlapping Intervals" (#435) కి **నిజంగా అదే problem** — arrows count = total – non-overlapping కి removals. End-sort greedy రెండింటికీ.
- Greedy "ఎందుకు end?" reasoning ని మాటల్లో చెప్పగలగడం SSE interview లో ప్లస్.

సాధారణ తప్పులు

- **Start బట్టి sort చేయడం** — greedy తప్పు అవుతుంది (కొన్ని cases లో ఎక్కువ arrows లెక్కిస్తుంది). **End బట్టి** తప్పనిసరి (లేదా start-sort + వేరే logic).
- **$> \text{vs } \geq$** : touching balloons ($[1,2]$ & $[2,3]$) ఒకే arrow ($x=2$) తో పోతాయి — **$\text{start} > \text{arrowPos}$** (strict) వాడాలి. $\geq$ వాడితే వాటిని విడదీసి ఎక్కువ arrows లెక్కిస్తుంది (తప్పు).
- **Empty array** — $\text{points}[0]$ access చేసేముందు $\text{length} === 0$ check. లేకపోతే **`undefined[1]`** crash.
- **$a[1]-b[1]$ overflow (ఇతర భాషల్లో)** — JS లో OK, కానీ Java/C++ కి safe comparator వాడాలి అని గుర్తుంచుకో (interview cross-language ప్రశ్న).

ముగింపు — Pattern Cheat-Sheet (ఒక్క చూపులో)

interview లో problem చదవగానే ఏ pattern అని గుర్తుపట్టడమే అసలు నైపుణ్యం. ఈ signals మనసులో ఉంచుకో:

Hash Map / Set వాడు — ఎప్పుడు?

సూచన (signal)	Technique	Problems
"count / frequency / ఎన్నిసార్లు"	frequency Map	Ransom Note, Valid Anagram
"duplicate / seen / repeat / cycle"	seen-Set	Contains Duplicate II, Happy Number
"కలిపితే target"	complement trick	Two Sum
"one-to-one / consistent mapping"	రెండు Maps (bijection)	Isomorphic, Word Pattern
"ఒకే లక్షణం ఉన్నవి group చెయ్"	key → list Map	Group Anagrams
"O(n) లో sorted-లాంటి పని"	Set + smart start	Longest Consecutive Sequence

Intervals వాడు – ఎప్పుడు?

సూచన (signal)	Technique	Problems
[start, end] array + "merge / overlap"	sort by start + merge	Merge Intervals, Insert Interval
"minimum arrows/points/rooms"	sort by end + greedy	Burst Balloons
"sorted list లో insert"	three-phase (left/merge/right)	Insert Interval
"వరుస numbers → ranges"	single-pass grouping	Summary Ranges

రెండు బంగారు నియమాలు:

1. **Hash map = space ఖర్చు పెట్టి time కొను.** Nested loop లోని "ఇది ఉందా?" search ని $O(1)$ lookup తో మార్పు → $O(n^2)$ నుండి $O(n)$.
2. **Interval problem = sort మొదటి అడుగు.** "merge" అయితే start బట్టి, "minimum/maximum non-overlapping" అయితే end బట్టి sort. Sort key ఎంపికే సగం పరిష్కారం.

చివరి మాట: ఈ 13 problems solutions కంటే – వాటి వెనుక ఉన్న **6 hashing signals + 4 interval signals** ముఖ్యం. Interview లో exact problem రాకపోవచ్చు, కానీ *signal* మళ్ళీ వస్తుంది. Signal కనిపిస్తే, template గుర్తిస్తుంది; template ఉంటే, solution వచ్చేస్తుంది. అదే "ఒకసారి చదివితే మర్చిపోకూడదు."